

Delegação Capacitária com Profundidade Limitada

Um modelo de controle de acesso distribuído sobre CASL,
com grafo de delegação, intensidade de propagação,
e *enforcement* por *Row-Level Security*

Patria Technology — patria-erp

Maio de 2026

Resumo

Modelamos o controle de acesso do **patria-erp** como *delegação capacitária*: autoridades intrínsecas (o operador da plataforma, que tudo vê; e cada *tenant*, com controle total dos próprios recursos) emitem *capacidades* que fluem por um **grafo de delegação** — não uma árvore: um concedente pode autorizar *diretamente* qualquer principal. Duas teses estruturam o modelo: (i) **desacoplar o *tenant***, que deixa de ser eixo especial e vira apenas *uma condição* dentro de uma capacidade; e (ii) dotar cada concessão de uma **intensidade** δ — a *profundidade de propagação* — que limita quantas re-delegações ela admite ($\delta = 0$: o destinatário usa mas não compartilha; $\delta = k$: re-delegável por mais k saltos, decrementando; $\delta = \infty$: irrestrito). Mostramos que a delegação é uma *atenuação* (condição $\subseteq$ e δ decrementado), o que garante não-escalada de privilégio por construção, e que o *enforcement* se reduz a empurrar a *união* das condições detidas para o **where** das consultas (*pushdown*), realizado por uma extensão do Prisma Client. Um ponto operacional central: δ é verificado no **ato de delegar**, não na leitura. O Apêndice A traz o plano de implementação.

Sumário

1	Introdução e motivação	1
2	Preliminares	2
3	Autorização de um principal	2
4	O grafo de delegação e a atenuação	3
4.1	A intensidade δ : profundidade de propagação	3
5	Invariantes de segurança	4

6	O tenant como caso particular (desacoplamento)	5
7	<i>Enforcement: pushdown no data layer</i>	5
A	Plano de implementação	6

1 Introdução e motivação

A Patria opera uma plataforma multi-organização. Cada cliente (um *tenant*) controla os próprios cadastros, vendas, alunos, mensalidades; a Patria, como operadora, precisa *ver e operar* centralmente sobre os clientes; e dentro de cada cliente há níveis de acesso. Isso é *delegação*: a autoridade flui de quem a tem para quem a recebe.

Duas decisões de projeto. Primeiro, **o *tenant* não é especial**: em vez de um “pisso” fixo (**where**:{**tenantId**}) espalhado por centenas de *services*, o isolamento é apenas a condição $c_t = (\text{tenantId} = t)$ embutida nas capacidades. Segundo — e é o ponto que distingue este modelo — toda concessão carrega uma **intensidade** δ que limita a sua re-delegação. Sem isso, dar a um usuário acesso a dados de outro *tenant* contaminaria toda a cadeia abaixo dele; com $\delta = 0$, o acesso fica *só naquele usuário*.

Por fim, a estrutura de delegação **não é uma árvore**: um *tenant* pode conceder uma permissão *diretamente* a um usuário específico. O substrato é um *grafo acíclico dirigido* (DAG) de concessões.

2 Preliminares

Definição 1 (Linhas e condições). Seja $\mathcal{R}$ o universo de todas as linhas. Cada $r \in \mathcal{R}$ tem atributos ($r.\text{tenantId}, r.\text{id}, \dots$). Uma *condição* c é um predicado $c : \mathcal{R} \rightarrow \{\perp, \top\}$, identificado ao seu *extent* $\llbracket c \rrbracket = \{r \in \mathcal{R} : c(r)\} \subseteq \mathcal{R}$. As condições formam a álgebra booleana $(2^{\mathcal{R}}, \cap, \cup, \setminus, \emptyset, \mathcal{R})$ — as *MongoQuery* do CASL.

Fixamos *subjects* $\mathcal{S}$ (**Customer**, **Student**, $\dots$), *ações* $\mathcal{A} = \{\text{create}, \text{read}, \text{update}, \text{delete}\}$ (**manage** = $\bigvee \mathcal{A}$) e o curinga **all**.

Definição 2 (Capacidade). Uma *capacidade* é uma tupla

$$\kappa = (a, s, c, \iota, \delta), \quad \delta \in \mathbb{N}_0 \cup \{\infty\},$$

com ação a , subject s , condição c , *flag* de inversão ι ($\iota=1$ é uma *proibição*, **cannot**) e *profundidade de propagação* (intensidade) δ . Informalmente, κ autoriza (ou nega, se $\iota=1$) a sobre s nas linhas de $\llbracket c \rrbracket$, e admite ser re-delegada por mais δ saltos.

3 Autorização de um principal

Seja P o conjunto de *principals* (operador, donos de *tenant*, perfis, usuários). Cada principal *detém* um conjunto de capacidades $\text{Hold}(p)$ (Seção 4). Para um par fixo (s, a) , a autorização efetiva é a **união** dos grants menos as proibições:

$$E_p(s, a) = \left(\bigcup_{\substack{\kappa \in \text{Hold}(p) \\ \iota_\kappa = 0, \kappa \vdash (s, a)}} \llbracket c_\kappa \rrbracket \right) \setminus \left(\bigcup_{\substack{\kappa \in \text{Hold}(p) \\ \iota_\kappa = 1, \kappa \vdash (s, a)}} \llbracket c_\kappa \rrbracket \right),$$

onde $\kappa \vdash (s, a)$ significa que κ casa o subject e a ação (com **manage**/**all** como curingas).

Observação 3 (A intensidade não entra aqui). E_p não depende de δ : a profundidade governa a *delegação* (Seção 4), não a avaliação. Quem detém uma capacidade a exerce plenamente, qualquer que seja o seu δ restante. Isso é o que torna o *enforcement* barato (Seção 7).

Observação 4 (União, não interseção). Compor grants com interseção seria um erro: “ler onde **org**= A ” e “ler onde **org**= B ” devem autorizar $A \cup B$. Só as proibições subtraem.

4 O grafo de delegação e a atenuação

Definição 5 (Autoridades intrínsecas). Duas fontes de autoridade não derivam de ninguém:

- o **operador** Ω detém $(\text{manage}, \text{all}, \top, 0, \infty)$ — vê e faz tudo, com re-delegação irrestrita;
- o **dono do tenant** t detém $(\text{manage}, \text{all}, (\text{tenantId}=t), 0, \infty)$ — controle total dos próprios recursos.

Definição 6 (Grafo de delegação). Um *grafo de delegação* é um DAG cujos vértices são principals e cujas arestas $u \xrightarrow{\kappa} w$ representam “ u concede a capacidade κ a w ”. Note que w é *qualquer* principal — não há exigência de árvore nem de vizinhança: o dono de um *tenant* pode conceder *diretamente* a um usuário específico.

Definição 7 (Concessão válida / atenuação). A aresta $u \xrightarrow{\kappa} w$ é *válida* se existe $\kappa_0 \in \text{Hold}(u)$ tal que

$$\llbracket c_\kappa \rrbracket \subseteq \llbracket c_{\kappa_0} \rrbracket, \quad \kappa \vdash (s, a) \Rightarrow \kappa_0 \vdash (s, a), \quad \iota_\kappa = \iota_{\kappa_0}, \quad \delta_{\kappa_0} \geq 1, \quad \delta_\kappa \leq \delta_{\kappa_0} - 1,$$

com a convenção $\infty - 1 = \infty$. Isto é, u só concede o que detém, *nunca ampliando* a condição e *sempre decrementando* a profundidade. Em particular $\delta_{\kappa_0} = 0$ proíbe qualquer concessão.

Definição 8 (Capacidades detidas). $\text{Hold}(p)$ é o menor conjunto que contém as capacidades intrínsecas de p (se p é Ω ou dono) e toda κ tal que existe uma aresta válida $u \xrightarrow{\kappa} p$.

4.1 A intensidade δ : profundidade de propagação

Por construção (Definição 7), uma capacidade emitida com profundidade δ admite cadeias de re-delegação de comprimento no máximo δ :

$$\delta \xrightarrow{1 \text{ salto}} \delta - 1 \rightarrow \dots \rightarrow 0 \text{ (não re-delegável)}.$$

- $\delta = 0$: o destinatário *usa* a capacidade, mas não a passa a ninguém. É o caso do “dar acesso sem permitir compartilhar”.
- $\delta = k$: re-delegável por mais k saltos; cada salto decremente.
- $\delta = \infty$: irrestrito (o comportamento legado, sem limite).

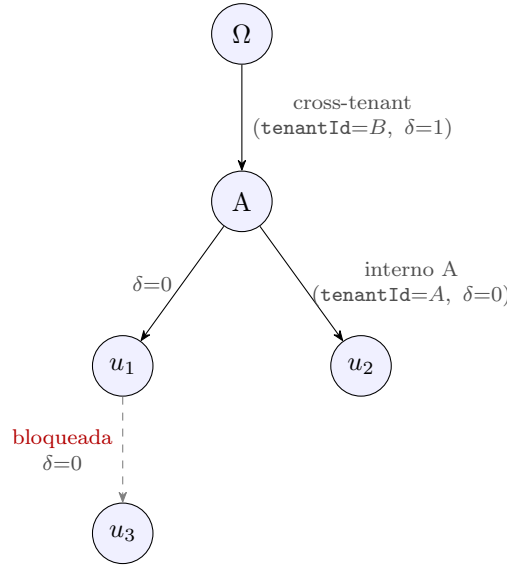


Figura 1: Grafo de delegação (DAG, não árvore). Ω concede a A uma capacidade *cross-tenant* (`tenantId=B`) com $\delta=1$; A pode re-delegá-la *uma* vez (a u_1 , com $\delta=0$), mas u_1 não pode repassá-la (aresta bloqueada). A também concede algo interno diretamente a u_2 . As arestas são explícitas — A alcança u_1, u_2 diretamente, sem intermediários obrigatórios.

5 Invariantes de segurança

Teorema 9 (Não-escalada por atenuação). *Para todo p e toda $\kappa \in \text{Hold}(p)$, existe uma autoridade intrínseca κ_0 e uma cadeia de arestas válidas de sua origem até p tais que $\llbracket c_\kappa \rrbracket \subseteq \llbracket c_{\kappa_0} \rrbracket$. Logo*

$$E_p(s, a) \subseteq \bigcup_{\kappa_0 \text{ (raiz que alcança } p)} \llbracket c_{\kappa_0} \rrbracket.$$

Nenhum principal acessa uma linha que nenhuma autoridade intrínseca em sua cadeia podia conceder.

Demonstração. Indução no comprimento da cadeia de delegação. Caso base: capacidade intrínseca. Passo: a Definição 7 exige $\llbracket c_\kappa \rrbracket \subseteq \llbracket c_{\kappa_0} \rrbracket$ a cada aresta; a inclusão é transitiva. A cota de E_p segue por monotonicidade da união. $\square$

Teorema 10 (Propagação limitada). *Uma capacidade emitida com profundidade δ só pode aparecer (atenuada) ao fim de cadeias de re-delegação de comprimento $\leq \delta$. Em particular, $\delta = 0 \Rightarrow$ não re-delegável: ela permanece exclusivamente no destinatário direto.*

Demonstração. Cada aresta válida decrementa δ em pelo menos 1 e exige $\delta_{\kappa_0} \geq 1$ (Definição 7); uma cadeia de comprimento ℓ requer $\delta \geq \ell$. Para $\delta = 0$ não há aresta de saída possível. $\square$

Corolário 11 (Compartilhamento controlado). *Conceder uma capacidade cross-tenant com $\delta = 0$ garante que o destinatário a exerce mas ninguém abaixo dele a herda — exatamente “dar acesso sem propagar”. Com $\delta = k$, a difusão é confinada à vizinhança de re-delegação de raio k .*

Observação 12 (Fail-closed = união vazia). Se nenhum grant detido casa (s, a) , então $E_p(s, a) = \emptyset$: acesso negado. É o *bottom* da álgebra, uniforme — sem piso de tenant especial.

6 O tenant como caso particular (desacoplamento)

Corolário 13 (Isolamento de tenant). *Um usuário comum da organização t só detém capacidades atenuadas da capacidade intrínseca do dono, todas com $\llbracket c \rrbracket \subseteq \llbracket \text{tenantId}=t \rrbracket$. Logo $E_p \subseteq \{r : r.\text{tenantId} = t\}$: nenhuma linha de outro tenant aparece, sem regra especial. Um vazamento cross-tenant só ocorre por uma aresta de concessão explícita (auditável), cuja difusão é exatamente governada por δ .*

O operador Ω , detendo $\top$, enxerga todos os *tenants* — “eu, no topo, vejo tudo”. O *tenant* foi reduzido a uma condição.

7 Enforcement: pushdown no data layer

A avaliação independe de δ (Seção 4): basta empurrar $E_p(s, a)$ para o **where**. Numa leitura de subject s pelo principal p ,

$$\text{retornado} = E_p(s, \text{read}) \cap \llbracket c_{\text{query}} \rrbracket.$$

Escritas validadas contra $E_p(s, a)$; o conjunto de campos restringe colunas. Concretamente: uma *extensão do Prisma Client* (`$allOperations`) que injeta $\bigcup(\text{grants}) \setminus \bigcup(\text{proibições})$ via AND.

Observação 14 (Onde δ é verificado). A intensidade é checada no **ato de delegar** — quando um principal cria/atribui uma capacidade a outro, valida-se a atenuação (Definição 7: condição $\subseteq$, $\delta' \leq \delta - 1$). A leitura de dados nunca consulta δ . Isso mantém a extensão RLS simples e a auditoria concentrada no momento da concessão.

Observação 15 (Dois modos e o piso nativo). **shadow** computa e *registra* onde negaria sem alterar o resultado; **enforce** aplica. **\$queryRaw** escapa da extensão de aplicação; por isso um teste-*guardrail* barra *raw* fora de uma *allowlist* escopada, e a Fase B adiciona *Row-Level Security* nativa do PostgreSQL como piso duro.

A Plano de implementação

Escopo (backbone). A avaliação (*pushdown* da união de capacidades detidas) já está implementada e testada (`condition-resolver.ts`, `prisma-rls.extension.ts`): ela *independe* de δ , logo vale desde já. A camada de *delegação* com δ (grafo, atenuação) entra com o multi-org; o campo e o primitivo de atenuação são preparados agora.

A1. Capacidade carrega δ . Adicionar `propagationDepth Int?` a `AccessRule` (`null = \infty`; `0 =` não re-delegável) e a `RuleInput`. Migração nullable, retro-compatível (regras atuais = ∞).

A2. Primitivo de atenuação (testado). `access/delegation.ts: attenuatedDepth(holder, proposed)` ($\infty - 1 = \infty$; exige `holder  $\geq$  1`; `proposed  $\leq$  holder - 1`) e `canDelegate`. Verificado no serviço de concessão (criar regra / atribuir perfil), *não* na leitura.

A3. Avaliação (já feita). `condition-resolver.ts:  $E_p$` = união dos grants \ proibições, sob o escopo herdado. `prisma-rls.extension.ts: injeção no where`, OR nos grants, fail-closed, guarda de *create* cross-escopo, modos **shadow/enforce**. Mapa *Subject* $\leftrightarrow$ *model*; *registry* via DMMF.

A4. Wiring. `TenantContext` carrega regras + escopo + `rlsMode`; `PrismaService` vira *Proxy* por-*request* com client estendido; `PoliciesGuard` reusa as regras do contexto.

A5. Testes (por funcionalidade — exigência central). `condition-resolver.spec`, `prisma-rls.extension.spec` (por subject/model), `delegation.spec` (atenuação/ δ), `ability.contract.spec` (matriz dos perfis), *guardrail* de *raw*. Regra de projeto: nenhuma capacidade/perfil novo sem teste.

A6. Gate na CI. `job test (npm ci + prisma generate + npm test);`
deploy com needs: `test`.

A7. Rollout. `RLS_MODE=shadow` → `enforce`. Fase B: RLS nativa do PostgreSQL como piso duro.